

Spring Boot Book Management Application Report

1. Introduction

This report details the development of a Spring Boot application designed to manage information for two entities: **Books** and **Authors**. The application implements core CRUD (Create, Read, Update, Delete) operations, utilizes JSP for the view layer, and integrates with an H2 in-memory database via JPA. This document outlines the project's architecture, implementation specifics, and challenges encountered during development.

2. Entity Relationship Design

The application models two primary entities: `Author` and `Book`. A one-to-many relationship exists between `Author` and `Book`, meaning one author can write multiple books, but each book is associated with only one author.

Author Entity

- `id` : Primary key, auto-generated.
- `name` : Name of the author (non-nullable).
- `nationality` : Nationality of the author.
- `books` : A list of books written by the author (One-to-Many relationship).

Book Entity

- `id` : Primary key, auto-generated.
- `title` : Title of the book (non-nullable).
- `isbn` : International Standard Book Number.

- `price` : Price of the book.
- `author` : The author of the book (Many-to-One relationship, non-nullable).

3. Implementation Details

3.1. Project Structure

The project follows a standard Spring Boot application structure, separating concerns into distinct layers:

- `com.example.bookmgmt.model` : Contains JPA entities (`Author.java` , `Book.java`).
- `com.example.bookmgmt.repository` : Defines data access interfaces (`AuthorRepository.java` , `BookRepository.java`) extending `JpaRepository` .
- `com.example.bookmgmt.service` : Implements business logic (`BookService.java` , `DataInitializer.java`).
- `com.example.bookmgmt.controller` : Handles HTTP requests and responses (`BookController.java` , `HomeController.java`).
- `src/main/webapp/WEB-INF/jsp` : Stores JSP view templates (`book-list.jsp` , `book-form.jsp`).
- `src/main/resources` : Contains application configuration (`application.properties`).

3.2. Populate Database (Data Initialization)

Upon application startup, the `DataInitializer` class (implementing `CommandLineRunner`) populates the H2 database with 10 sample authors and 10 corresponding books. This ensures that there is initial data available for testing and demonstration purposes.

3.3. Create Operation

- **JSP Form:** `book-form.jsp` provides a user interface to add new books. It includes input fields for title, ISBN, price, and a dropdown to select an existing

author.

- **Controller Method:** The `BookController`'s `@PostMapping("/save")` method handles form submissions. It binds the form data to a `Book` object and uses `BookService` to persist it. Exception handling is included to catch potential integrity violations.

3.5. Read Operation

- **JSP View:** `book-list.jsp` displays a tabular list of all books, including their titles, ISBNs, prices, and associated author details.
- **Controller Method:** The `BookController`'s `@GetMapping("/books")` method fetches all books using `bookService.getAllBooks()` and adds them to the model for rendering in the JSP.
- **Custom Query:** The `BookRepository` includes a custom JPA query `findAllBooksWithAuthors()` that performs an inner join between `Book` and `Author` entities to fetch all books along with their author information efficiently.

3.6. Update Operation

- **JSP Form:** The same `book-form.jsp` is reused for updating. When an "Edit" link is clicked from the book list, the `BookController`'s `@GetMapping("/edit/{id}")` method retrieves the existing book's details and pre-populates the form fields.
- **Controller Method:** The `@PostMapping("/save")` method handles both create and update operations. If the `book` object has an `id`, it's treated as an update; otherwise, it's a new creation.

4. Challenges Faced and Solutions

Challenge: Initial compilation failed due to a Java version mismatch between the project configuration (Java 17) and the environment's default settings.

Solution:

1. Verified the environment's Java capabilities and ensured the JDK was correctly configured.

2. Aligned the `pom.xml` configuration with the available Java 17 environment.
3. Successfully executed `mvn clean compile test` to verify the application's robustness and the correctness of the business logic.

5. Conclusion

This project successfully demonstrates the implementation of a Spring Boot application with JPA, H2 database, and JSP views for managing Books and Authors. It covers essential CRUD operations, proper layering, and includes unit tests for key components. The application provides a solid foundation for further development and showcases best practices in Spring Boot development.